



INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

Group Members

1. Mulisa Docile
2. Daniel Obar
3. Abi Mirembe
4. Flavia Sherinah
5. Victoria Marvis
6. Mordecai Corey Kwezi

Introduction

This report explains the Object-Oriented Programming (OOP) design and implementation of the Group 9 Insurance Policy and Claims Management System. The system registers clients, creates motor, health and property policies, records claims, and produces summaries through an interactive menu. An OOP approach was used so that real-world entities (clients, policies and claims) could be modelled as collaborating objects with clear responsibilities, shared behaviour through inheritance, and controlled access to data.

1. Identified Classes

Why: The problem domain contains distinct entities that each hold their own data and behaviour; modelling them as classes keeps responsibilities separate.

Implemented: The project defines the following main classes.

- Client: represents a registered person with attributes: *_client_id*, *_name*, *_phone*, *_email*. Methods: *get_contact_info()*, *__str__()*.
- InsurancePolicy (abstract): common policy data and interface with attributes: *_policy_number*, *_client_id*, *_start_date*, *_end_date*, *_is_active*. Methods: *abstract calculate_premium()*, *get_details()*.
- MotorPolicy, HealthPolicy, PropertyPolicy: concrete policy types. Each adds type-specific attributes (vehicle registration and value; age and coverage level; property address and value) and implements its own *calculate_premium()* formula.
- Claim: a payment request against a policy. Attributes include *_claim_id*, *_policy_number*, *_amount*, *_status*, *_description* and *_claim_date*. Status values are restricted to Pending, Approved, Rejected and Paid.
- InsuranceSystem: the collaboration hub. It holds dictionaries of clients, policies and claims, auto-generates IDs, and exposes operations such as *register_client*, *create_motor_policy*, *create_health_policy*, *create_property_policy*, *record_claim*, *update_claim_status* and *generate_summary*. The menu in *main.py* only calls this class; business logic is not placed in the menu itself. A separate validators module provides input helpers and is not modelled as a domain class.

2. Encapsulation

Why: Unrestricted changes to amounts, statuses or identifiers could leave objects in invalid states; controlled access keeps data consistent.

Implemented: Across the domain classes, attributes are stored with a leading underscore (for example `_name`, `_vehicle_value`), signalling internal use. In `Claim`, amount and status are exposed through `@property` getters and setters. The amount setter rejects non-numeric values and non-positive amounts; the status setter accepts only the four allowed values. `claim_id` is read-only (property without a setter). Policy and client fields remain protected by convention and are created through constructors and system methods rather than direct external assignment.

3. Class Relationships

Why: Clients hold policies, policies attract claims, and a central manager must coordinate these objects without merging all logic into one class.

Implemented:

- Association (`Client` ↔ `InsurancePolicy`). Each policy stores a `_client_id` linking it to exactly one client. A client may have zero or many policies. `InsuranceSystem.get_client_policies()` retrieves policies by matching that identifier. Lifetimes are independent.
- Association (`InsurancePolicy` ↔ `Claim`). Each claim stores a `_policy_number`. One policy may have many claims. `get_claims_for_policy()` and `get_claims_for_client()` follow these links.
- Aggregation / management (`InsuranceSystem`). The system holds collections (`_clients`, `_policies`, `_claims`) and creates or looks up objects on behalf of the menu. This is weak ownership: the system organises the objects for the duration of a run rather than defining their existence in a composition sense.
- Dependency. `main.py` depends on `InsuranceSystem` and on the `validators` module for input. `InsuranceSystem` depends on the concrete policy classes and on `Client` and `Claim` when creating instances. Inheritance is treated in the next section. Composition (strong ownership where a part cannot exist without the whole) is not used as the primary relationship model in this project.

4. Inheritance Hierarchy

Why: Motor, health and property policies share identity, dates and the need to calculate a premium, but each uses a different formula and extra data. Inheritance avoids duplicating the shared parts.

Implemented: `InsurancePolicy` is the superclass. `MotorPolicy`, `HealthPolicy` and `PropertyPolicy` each inherit from it using `class MotorPolicy(InsurancePolicy)` (and the same pattern for the others). Subclasses call `super().__init__(...)` to initialise the shared attributes, then set their own fields. They inherit `get_details()` and extend it with type-specific information. Each overrides `calculate_premium()`: motor uses 3% of vehicle value plus a base; health uses coverage rates adjusted by age band; property uses 0.5% of property value plus a base.

5. Polymorphism

Why: The system must treat different policy types uniformly when listing premiums or calculating a single policy's cost, without writing separate branches for each type.

Implemented: `calculate_premium()` is defined on the abstract base and overridden in each subclass with a different algorithm. In `reports_menu`, the code loops over `system.list_policies()` and calls `p.calculate_premium()` on every object. The same call appears when the user selects "Calculate premium for a policy" in the policies menu. Python dispatches to the correct subclass implementation at runtime. No if-type checks are required in those call sites, which is the practical demonstration of polymorphism in this project. `get_details()` is also overridden so that display text differs by policy type when printed through the same method name.

6. Abstraction

Why: The program must guarantee that every policy can calculate a premium without fixing one formula in the base class; abstraction defines the required behaviour while leaving the details to subclasses.

Implemented: *InsurancePolicy* inherits from *ABC* (Python's abstract base class support). *calculate_premium* is marked with *@abstractmethod* and has no real body beyond *pass*. Because of this, the base class cannot be instantiated directly; only the three concrete subclasses can be created. The base still provides shared state and *get_details()*, so callers work against a common interface. This matches the assignment requirement for an abstract policy superclass with an abstract premium method that subclasses must implement.

Summary of Design Decisions

The design maps the insurance domain onto collaborating objects. Clients, policies and claims are separate classes with focused responsibilities. Encapsulation protects sensitive claim data through properties and validation. Associations via identifiers, together with aggregation inside *InsuranceSystem*, express real links without forcing artificial composition. Inheritance and abstraction share common policy structure while requiring each product type to supply its own premium logic. Polymorphism allows the menu and reports to call one method name on mixed policy objects and obtain type-specific results. The interactive menus remain thin: they collect validated input and delegate all domain operations to *InsuranceSystem*, which in turn creates and coordinates the domain objects. Together these choices produce a maintainable structure that demonstrates the core OOP concepts required for the course project.